

12576 - Nisanth Saravanan

L1 Type Script Assessment

Real-Time Traffic Monitoring System

Case Study:

This system provides real-time traffic data to help authorities manage road congestion and inform drivers of traffic conditions.

Entity:

TrafficReport

Required Fields:

ReportId, Location, TrafficDensity, AverageSpeed, IncidentDetails

GitHub URL:

[Nisanth-Saravanan/L1_TypeScript_Assessment](#)

W3H Analysis

Project Title: Real-Time Traffic Monitoring System	
What?	How?
1.What are the Modules/Functionalities needed? A: Modules/Functionalities Required: a) Landing Component. b) Reporting Component. c) Display Functionality. d) Search Functionality. e) Delete Functionality. f) Update Functionality. 2. What are the requirements of Landing Component? A: Landing Component: a) Greet the user with a custom message. b) Redirect or Render in the Reporting Component. c) Return back to Landing Component. 3. What are the requirements of the Reporting Component? A: Reporting Component: a) Get User Input. b) Validate user input. c) Inject new data into the JSON file. 4. What are the requirements of the Display functionality? A: Display Functionality: a) Display user data. b) Provide means to search. c) Provide means to update.	1. Landing Component: a) Greet the user with a custom message: Method 1: Use simple container to greet the users in a landing page. Method 2: Greet users in an alert. Method 3: Have a series of prompts that lead the user to add a Reporting by default. b) Redirect or Render in the Reporting Component. Method 1: Have a dedicated Navigation Bar. Method 2: Have simple buttons to render in the component. Method 3: Have the user be automatically redirected to the component. c) Return back to Landing Component. Method 1: Have a dedicated Navigation Bar. Method 2: Have simple buttons to render in the component. Method 3: Have the user refresh the page to revisit the landing page. 2. Reporting Components: a) Get User Input: Method 1: Have a simple form. Method 2: Use a series of prompts. Method 3: Use modular forms with optional modules. b) Validate User Input: Method 1: User JS validations and alerts.

d) Provide means to delete.

5. What are the requirements of the Search functionality?

A: Search Functionality:

- a) To search the Reporting based on Reporting ID.
- b) To display search results.

6. What are the requirements of the Delete Functionality?

A: Delete Functionality:

- a) To delete the Reporting selected.

7. What are the requirements of the Update Functionality?

A: Update Functionality:

- a) To update the Reporting selected.
- b) To validate the update.

Method 2: Use default validations.

Method 3: Use JS validations and HTML elements.

c) Inject new data into JSON file:

Method 1: Use Async Axios.

Method 2: Use Redux.

Method 3: Use Sync Axios.

3. Display Functionality:

a) Display user data:

Method 1: Display as table.

Method 2: Display as cards.

Method 3: Display as detailed <div>'s

b) Provide means to search:

Method 1: Use a button and prompts.

Method 2: Use input field and ask id.

Method 3: Render a separate component.

c) Provide means to update:

Method 1: Use a button and the input field.

Method 2: Use input field and ask id.

Method 3: Render a separate component.

d) Provide means to delete:

Method 1: Use a button and alerts.

Method 2: Use input field and ask id.

Method 3: Render a separate component.

4. Search Functionality:

a) To search the Reporting based on Reporting ID:

Method 1: Use prompt and ask for unique Reporting ID.

Method 2: Use Input field and ask for unique Reporting ID.

Method 3: Use range filters.

b) To display search results:

Method 1: Use prompts.

Method 2: Use cards.

Method 3: Re-render the table.

5. Delete Functionality:

a) To delete the Reporting selected:

	Method 1: Use a simple button next to a Reporting. Method 2: Use input field and ask for id. Method 3: Use filters to select a Reporting. 6. Update Functionality: a) To update the Reporting selected: Method 1: Use prompts to get user input. Method 2: Use form's input field. Method 3: Render a separate component for updating. b) To validate the update: Method 1: User JS validations and alerts. Method 2: Render components to showcase validations. Method 3: Use JS validations and HTML elements.
1. Landing Component: a) Greet the user with a custom message: Method 1: Use simple container to greet the users in a landing page. A: Having to use a simple landing page give a soft welcome to our app. b) Redirect or Render in the Reporting Component. Method 1: Have a dedicated Navigation Bar. A: Gives an easy approach for accessing the needed component. c) Return back to Landing Component. Method 1: Have a dedicated Navigation Bar. A: Gives an easy approach for accessing the needed component. 2. Reporting Components: a) Get User Input:	1. Landing Component: a) Greet the user with a custom message: Greeting users in an alert and having a series of prompts are improper and over the top respectfully. b) Redirect or Render in the Reporting Component. A button-based navigation for a simple Reporting and displaying app will be underwhelming UX, and to have the auto redirect will affect the UX. c) Return back to Landing Component. A button-based navigation for a simple Reporting and displaying app will be underwhelming UX, and to have the auto redirect will affect the UX. 2. Reporting Components: a) Get User Input: Multiple prompts for every Reporting will affect the UX and modular forms are an

Method 1: Have a simple form.

A: Efficient and can be easily handled to our requirements.

b) Validate User Input:

Method 1: User JS validations and alerts.

A: Gives an efficient, yet effective means of validations.

c) Inject new data into JSON file:

Method 1: Use Async Axios.

A: The async axios gives an assurance to not lose data under varying data speeds.

3. Display Functionality:

a) Display user data:

Method 1: Display as table.

A: Provides a clean UI that can be easily manipulated to our requirements.

b) Provide means to search:

Method 1: Use a button and prompts.

A: Provides a clean UI that can be easily manipulated to our requirements, and still have the needed functionality.

c) Provide means to update:

Method 1: Use a button and the input field.

A: Provides a clean UI that can be easily manipulated to our requirements, and still have the needed functionality.

d) Provide means to delete:

Method 1: Use a button and alerts.

A: Provides a clean UI that can be easily manipulated to our requirements, and still have the needed functionality.

4. Search Functionality:

overkill for a simple requirement.

b) Validate User Input:

Default validations can not state out need and using HTML elements will increase on screen elements but can be implemented if project gains complexity.

c) Inject new data into JSON file:

Redux has a complex state management and is not needed here, and sync axios, doesn't provide the "Promise" that the data won't be lost under fluctuating network speed.

3. Display Functionality:

a) Display user data:

Card-like and separate division add in a lot of spaces in our UI.

b) Provide means to search:

Since the Reporting id is unique, a button can work effectively and have a very lightweight code behind its working.

d) Provide means to delete:

Since the Reporting id is unique, a button can work effectively and have a very lightweight code behind its working.

c) Provide means to update:

Since the Reporting id is unique, a button can work effectively and have a very lightweight code behind its working.

4. Search Functionality:

a) To search the Reporting based on Reporting ID:

Since the Reporting id is unique, a prompt can work effectively and have a very lightweight code behind its working.

a) To search the Reporting based on Reporting ID: Method 1: Use prompt and ask for unique Reporting ID. A: Provides a clean UI that can be easily manipulated to our requirements, and still have the needed functionality. b) To display search results: Method 1: Use prompts. A: Since the Reporting id is unique, only a single Reporting will be returned, and so a prompt will be sufficient. 5.Delete Functionality: a) To delete the Reporting selected: Method 1: Use a simple button next to a Reporting. A: Provides a clean UI that can be easily manipulated to our requirements, and still have the needed functionality. 6. Update Functionality: a) To update the Reporting selected: Method 1: Use prompts to get user input. A: Effective way to get new data and perform validations. b) To validate the update: Method 1: User JS validations and alerts. A: Gives an efficient, yet effective means of validations.	b) To display search results: Since the Reporting id is unique, only a single Reporting will be returned, and so a prompt will be sufficient. 5.Delete Functionality: a) To delete the Reporting selected: Since the Reporting id is unique, a button can work effectively and have a very lightweight code behind its working. 6. Update Functionality: a) To update the Reporting selected: A: With a low complexity, using HTML elements for each updates just adds more components, and a separate component is inefficient. b) To validate the update: Using HTML elements will increase on screen elements but can be implemented if project gains complexity and rendering components is an overkill.
Why?	Why Not?

Code:

App.tsx

```
import React from "react";
import { BrowserRouter, Routes, Route } from "react-router-dom";
import Navbar from "../components/NavBar";
import Home from "../components/Home";
import ReportList from "../components/ReportList";
import AboutUs from "../components/AboutUs";
import "../App.css";

function App() {
  return (
    <BrowserRouter>
      <Navbar />
      <div className="App">
        <header className="App-header">
          <Routes>
            <Route path="/" element={<Home />} />
            <Route path="/reports" element={<ReportList />} />
            <Route path="/about-us" element={<AboutUs />} />
          </Routes>
        </header>
      </div>
    </BrowserRouter>
  );
}

export default App;
```

Navbar.tsx

```
import { Link, useLocation } from "react-router-dom";
import "./Navbar.css";

function Navbar() {
  const location = useLocation();

  return (
    <nav
      className="navbar navbar-expand-lg bg-body-tertiary"
      style={{ backgroundColor: "#7AE2CF" }}
    >
      <div className="container-fluid">
        <Link className="navbar-brand" to="/">
          Traffic Reports
        </Link>
        <div className="collapse navbar-collapse" id="navbarNav">
          <ul className="navbar-nav">
            <li className="nav-item">
              <Link
                className={`nav-link ${
                  location.pathname === "/" ? "active" : ""
                }`}
                aria-current="page"
                to="/"
              >
                Home
              </Link>
            </li>
            <li className="nav-item">
              <Link
                className={`nav-link ${
                  location.pathname === "/reports" ? "active" : ""
                }`}
                aria-current="page"
                to="/reports"
              >
                Reports
              </Link>
            </li>
            <li className="nav-item">
              <Link
                className={`nav-link ${
                  location.pathname === "/about-us" ? "active" : ""
                }`}
                aria-current="page"
                to="/about-us"
              >
```



```
        About Us
      </Link>
    </li>
  </ul>
</div>
</div>
</nav>
);
}

export default Navbar;
```

ReportForm.tsx

```
import React from "react";
import { useState, useEffect } from "react";
import "./ReportForm.css";

interface ReportFormProps {
  fetchReports: () => void;
  isEditing: boolean;
  editId: number | null;
  initialData: {
    reportId: number | "";
    reportLocation: string;
    trafficDensity: string;
    averageSpeed: number | "";
    incidentDetails: string;
  };
  handleUpdate: (id: number, empData: any) => Promise<void>;
  handleCreate: (empData: any) => Promise<void>;
}

const ReportForm: React.FC<ReportFormProps> = ({
  fetchReports,
  isEditing,
  editId,
  initialData = {
    reportId: "",
    reportLocation: "",
    trafficDensity: "",
    averageSpeed: "",
    incidentDetails: "",
  },
  handleUpdate,
  handleCreate,
}) => {
  const [reportId, setReportId] = useState(initialData.reportId);
  const [reportLocation, setReportLocation] = useState(
    initialData.reportLocation
  );
  const [trafficDensity, setTrafficDensity] = useState(
    initialData.trafficDensity
  );
  const [averageSpeed, setAverageSpeed] = useState(initialData.averageSpeed);
  const [incidentDetails, setIncidentDetails] = useState(
    initialData.incidentDetails
  );

  useEffect(() => {
    setReportId(initialData.reportId || "");
```

```
setReportLocation(initialData.reportLocation);
setTrafficDensity(initialData.trafficDensity);
setAverageSpeed(initialData.averageSpeed || "");
setIncidentDetails(initialData.incidentDetails);
}, [initialData]);
```

```
const handleSubmit = async (e: React.FormEvent) => {
  e.preventDefault();
```

```
  if (isEditing && editId !== null) {
    await handleUpdate(editId, {
      reportId,
      reportLocation,
      trafficDensity,
      averageSpeed,
      incidentDetails,
    });
```

```
  } else {
    await handleCreate({
      reportId,
      reportLocation,
      trafficDensity,
      averageSpeed,
      incidentDetails,
    });
```

```
  }
  fetchReports();
```

```
  setReportId("");
  setReportLocation("");
  setTrafficDensity("");
  setAverageSpeed("");
  setIncidentDetails("");
};
```

```
return (
  <div className="form-container">
    <form onSubmit={handleSubmit} noValidate>
      <div className="form-group">
        <table>
          <tbody>
            <tr>
              <td className="label">
                <label htmlFor="reportId">Report ID (in 100s):</label>
              </td>
              <td>
                <input
                  type="number"
                  id="reportId"
                />
              </td>
            </tr>
          </tbody>
        </table>
      </div>
    </form>
  </div>
);
```

```

        min={100}
        value={reportId}
        onChange={(e) => setReportId(parseInt(e.target.value))}
      />
    </td>
  </tr>
  <tr>
    <td className="label">
      <label htmlFor="reportLocation">Report Location:</label>
    </td>
    <td>
      <input
        type="text"
        id="reportLocation"
        value={reportLocation}
        onChange={(e) => setReportLocation(e.target.value)}
      />
    </td>
  </tr>
  <tr>
    <td className="label">
      <label htmlFor="trafficDensity">Traffic Density:</label>
    </td>
    <td>
      <select
        id="trafficDensity"
        defaultValue=""
        value={trafficDensity}
        onChange={(e) => setTrafficDensity(e.target.value)}
      >
        <option value="">Select Traffic Density</option>
        <option value="Light">Light</option>
        <option value="Moderate">Moderate</option>
        <option value="Heavy">Heavy</option>
      </select>
    </td>
  </tr>
  <tr>
    <td className="label">
      <label htmlFor="averageSpeed">Average Speed (in KMPH):</label>
    </td>
    <td>
      <input
        type="number"
        id="averageSpeed"
        min={0}
        value={averageSpeed}
        onChange={(e) =>
          setAverageSpeed(parseFloat(e.target.value))
        }
      />
    </td>
  </tr>

```

```

    }
  />
</td>
</tr>
<tr>
  <td className="label">
    <label htmlFor="incidentDetails">Incident Details:</label>
  </td>
  <td>
    <input
      type="text"
      id="incidentDetails"
      value={incidentDetails}
      onChange={(e) => setIncidentDetails(e.target.value)}
    />
  </td>
</tr>
<tr>
  <td colspan={4}>
    <button className="submit-button" type="submit">
      {isEditing ? "Update" : "Create"}
    </button>
  </td>
</tr>
</tbody>
</table>
</div>
</form>
</div>
);
};

```

```
export default ReportForm;
```

ReportList.tsx

```
import React, { useState, useEffect } from "react";
import {
  fetchReports,
  deleteReport,
  updateReport,
  createReport,
} from "../services/api";
import ReportForm from "../ReportForm";
import "../ReportList.css";

const ReportList: React.FC = () => {
  const [reports, setReports] = useState<any[]>([]);
  const [editingReport, setEditingReport] = useState<any | null>(null);

  useEffect(() => {
    document.title = "Reports";
    fetchReports().then(setReports);
  }, []);

  const handleDelete = async (id: number) => {
    await deleteReport(id);
    fetchReports().then(setReports);
    alert("Report deleted successfully!");
  };

  const handleEdit = (Report: any) => {
    setEditingReport(Report);
  };

  const handleUpdate = async (id: number, report: any) => {
    if (report.reportId === id) {
      alert("Edit: Report ID cannot be the same as before!");
    } else if (report.reportId < 100) {
      alert("Edit: Report ID must be at least 100!");
    } else if (
      reports.map((report) => report.reportId).includes(report.reportId)
    ) {
      alert("Edit: Report ID already exists!");
    } else if (report.Location === "" || report.Location === null) {
      alert("Edit: Report Location cannot be empty!");
    } else if (report.trafficDensity === "" || report.trafficDensity === null) {
      alert("Traffic Density cannot be empty!");
    } else if (report.averageSpeed === "" || report.averageSpeed === null) {
      alert("Edit: Average Speed cannot be empty!");
    } else if (
      report.incidentDetails === "" ||
      report.incidentDetails === null
    ) {
      alert("Edit: Incident Details cannot be empty!");
    }
  };
}
```

```

) {
  alert("Edit: Incident Details cannot be empty!");
} else {
  try {
    await updateReport(id, report);
    fetchReports().then(setReports);
    setEditingReport(null);
    alert("Report updated successfully!");
  } catch (error) {
    console.error("Error updating report:", error);
    alert("Error updating report. Please try again.");
  }
}
};

```

```

const handleCreate = async (report: any) => {
  if (report.reportId === null) {
    alert("Report ID cannot be empty!");
  } else if (report.reportId < 100) {
    alert("Report ID must be at least 100!");
  } else if (
    reports.map((report) => report.reportId).includes(report.reportId)
  ) {
    alert("Report ID already exists!");
  } else if (report.Location === "" || report.Location === null) {
    alert("Report Location cannot be empty!");
  } else if (report.trafficDensity === "" || report.trafficDensity === null) {
    return alert("Traffic Density cannot be empty!");
  } else if (report.averageSpeed === "" || report.averageSpeed === null) {
    alert("Average Speed cannot be empty!");
  } else if (
    report.incidentDetails === "" ||
    report.incidentDetails === null
  ) {
    alert("Incident Details cannot be empty!");
  } else {
    try {
      await createReport(report);
      fetchReports().then(setReports);
      alert("Report created successfully!");
    } catch (error) {
      console.error("Error creating report:", error);
      alert("Error updating report. Please try again.");
    }
  }
};

```

```

const handleSearch = async () => {
  const reportId = prompt("Enter Report ID:");

```

```

if (reportId) {
  const result: any = reports.filter(
    (report) => report.reportId === parseInt(reportId)
  );
  if (result.length > 0) {
    result.forEach((report: any) => {
      alert(
        `Report Found! \n===== \nReport ID: ${report.reportId} \
nReport Location: ${report.reportLocation} \nTraffic Density: ${report.trafficDensity} \nAverage
Speed: ${report.averageSpeed} \nIncident Details: ${report.incidentDetails}`
      );
    });
  } else {
    alert("No report found with the provided Report ID.");
  }
} else {
  alert("Search: Cancelled!");
}
};

if (reports.length === 0) {
  return (
    <div className="fader">
      <h2>Report Form</h2>
      <ReportForm
        fetchReports={() => fetchReports().then(setReports)}
        isEditing={!editingReport}
        editId={editingReport !== null ? editingReport.id : null}
        initialData={
          editingReport || {
            reportId: 0,
            reportLocation: "",
            trafficDensity: 0,
            averageSpeed: 0,
            incidentDetails: "",
          }
        }
        handleUpdate={handleUpdate}
        handleCreate={handleCreate}
      />
      <br />
      <br />
      <h2>Report List</h2>
      <br />
      <p>No Reports found.</p>
      <br />
      <br />
    </div>
  );
}

```



```

);
} else {
  return (
    <div className="fader">
      <h2>Report Form</h2>
      <ReportForm
        fetchReports={() => fetchReports().then(setReports)}
        isEditing={!editingReport}
        editId={editingReport !== null ? editingReport.id : null}
        initialData={
          editingReport || {
            reportId: 0,
            reportLocation: "",
            trafficDensity: 0,
            averageSpeed: 0,
            incidentDetails: "",
          }
        }
        handleUpdate={handleUpdate}
        handleCreate={handleCreate}
      />
      <br />
      <br />
      <h2>Report List</h2>
      <br />
      <button className="search-button" onClick={handleSearch}>
        Search
      </button>
      <br />
      <br />
      <table className="user-table">
        <thead>
          <tr>
            <th>Report ID</th>
            <th>Report Location</th>
            <th>Traffic Density</th>
            <th>Average Speed</th>
            <th>Incident Details</th>
            <th>Actions</th>
          </tr>
        </thead>
        <tbody>
          {reports.map((report) => (
            <tr key={report.id}>
              <td>{report.reportId}</td>
              <td>{report.reportLocation}</td>
              <td>{report.trafficDensity}</td>
              <td>{report.averageSpeed} KMPH</td>
              <td>{report.incidentDetails}</td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  );
}

```

```
        <td>
          <button
            className="edit-button"
            onClick={() => handleEdit(report)}
          >
            Edit
          </button>
          <button
            className="delete-button"
            onClick={() => handleDelete(report.id)}
          >
            Delete
          </button>
        </td>
      </tr>
    )}
  </tbody>
</table>
<br />
<br />
</div>
);
}
};
```

```
export default ReportList;
```

Non-essential Codes:

Home.tsx

```
import React, {useEffect} from "react";

function Home() {
  useEffect(() => {
    document.title = "Home";
  })

  return (
    <div className="container fader">
      <h1>Welcome to Traffic Monitor</h1>
      <p>{`Welcome to Traffic Monitor, a simple app for monitoring traffic in different locations. ${
        Math.random() < 0.5
          ? "It's a great day to drive!"
          : "Be careful on the roads!"
        }`}</p>
      <br />
      <h4>What is Traffic Monitor?</h4>
      <p style={{ fontSize: "x-small" }}>
        Lorem ipsum dolor sit amet consectetur adipiscing elit. Quisque faucibus
        ex sapien vitae pellentesque sem placerat. In id cursus mi pretium
        tellus dui convallis. Tempus leo eu aenean sed diam urna tempor.
        Pulvinar vivamus fringilla lacus nec metus bibendum egestas. laculis
        massa nisl malesuada lacinia integer nunc posuere. Ut hendrerit semper
        vel class aptent taciti sociosqu. Ad litora torquent per conubia nostra
        inceptos himenaeos.
      </p>
    </div>
  );
}
```

export default Home;

AboutUs.tsx

```
import React, {useEffect} from "react";

function AboutUs() {
  useEffect(() => {
    document.title = "About Us";
  })

  return (
    <div className="container fader">
      <h1>About Us</h1>
      <br />
      <p>
        Traffic Monitor is a simple web application for monitoring traffic in
        different locations. It is a React application, built with Vite and
        TypeScript. The application uses a REST API to store and retrieve data.
        The API is mocked using json-server.
      </p>
      <p>
        The application is fully responsive and works on all devices with a
        modern web browser. The application is also fully accessible and follows
        the best practices for web accessibility.
      </p>
      <p>
        The application is licensed under the MIT License. The source code is
        available on GitHub.
      </p>
    </div>
  );
}

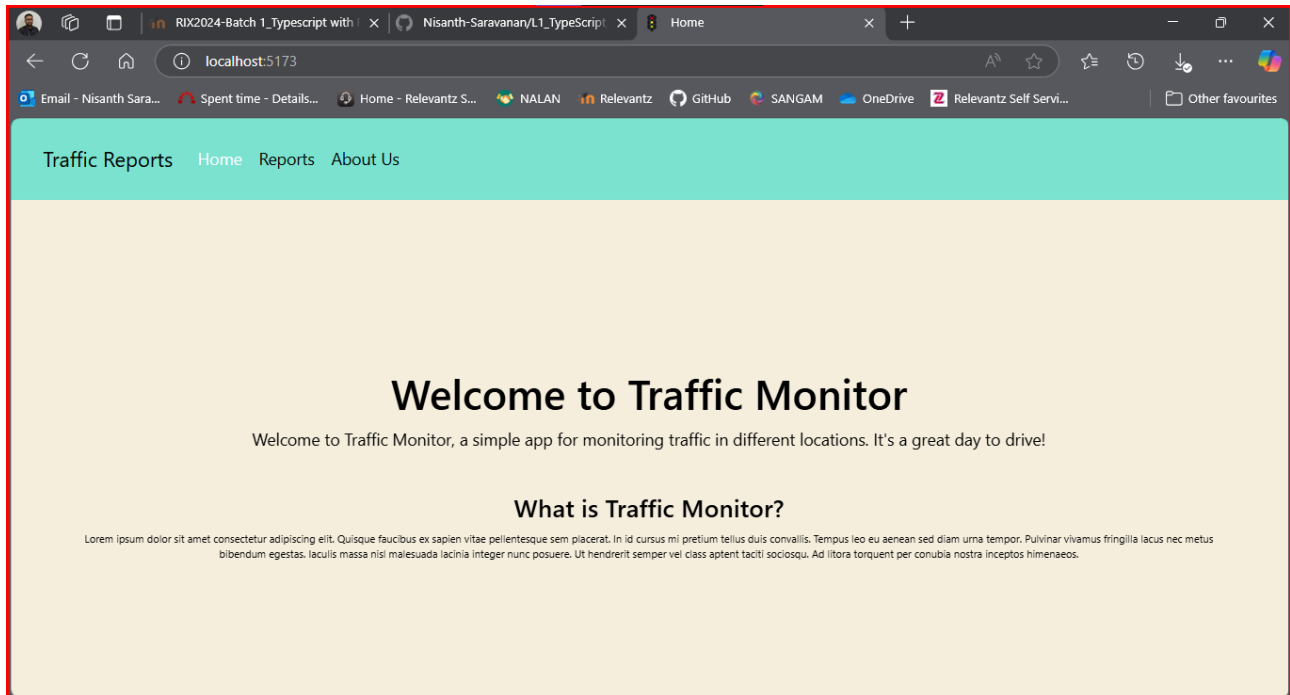
export default AboutUs;
```

Outputs:

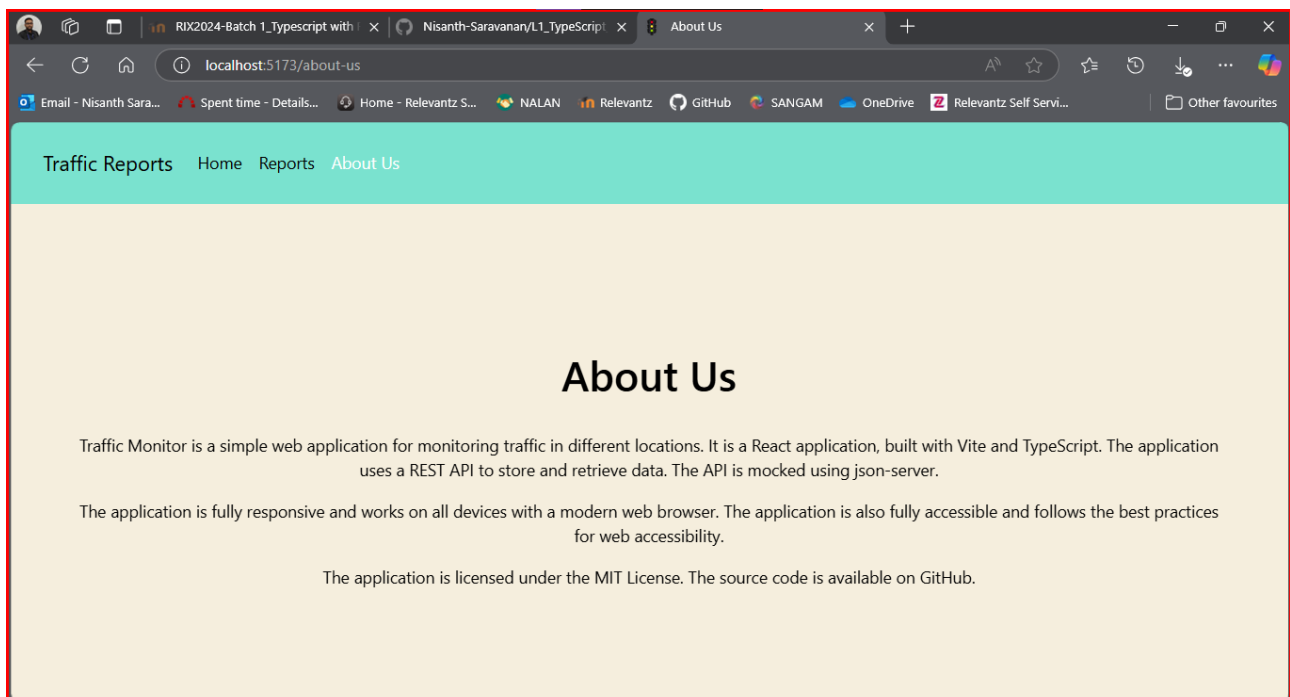
Key Notes:

- All report IDs must be unique.
- Requires axios, react-router-dom, and json-server.
- All fields have the basic level of validations.
- Creation and Editing both make use of the same input fields.
- All components are rendered in App.tsx with a fade-in animation using basic CSS animations.
- The active Nav-bar link will be white.
- The web page's title gets actively changed with respect to the active page/location.

Home Page

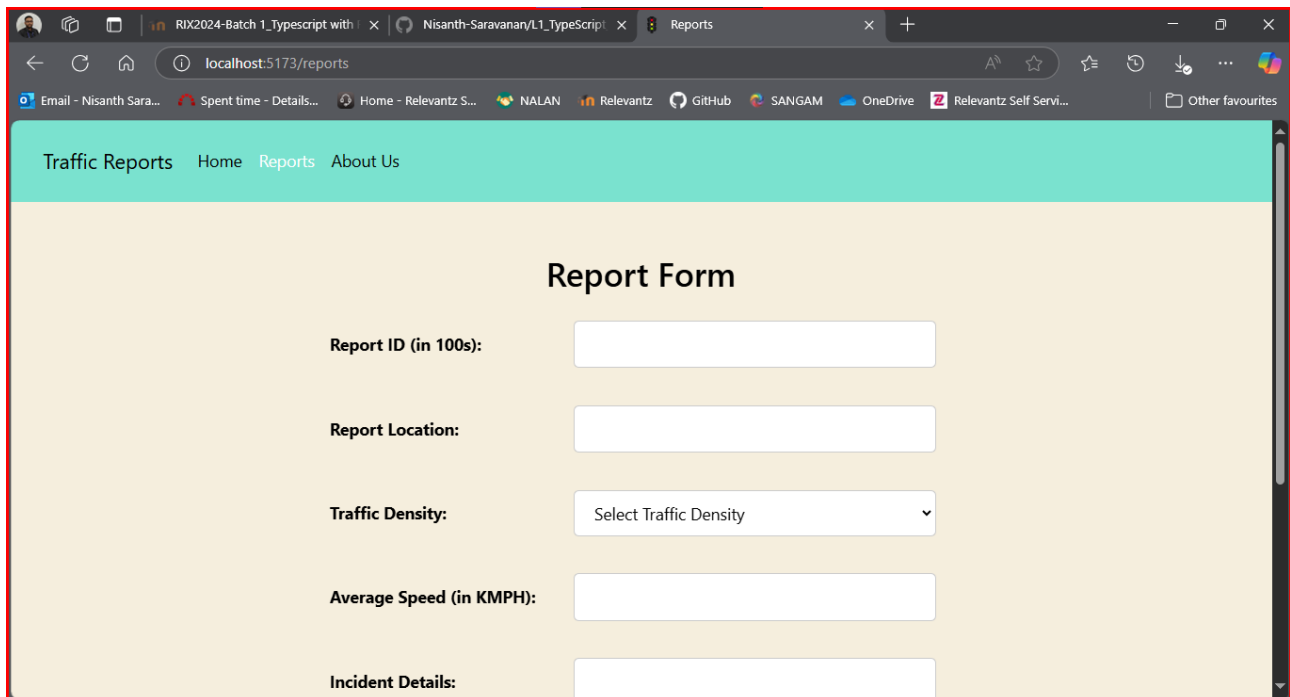


About Us Page

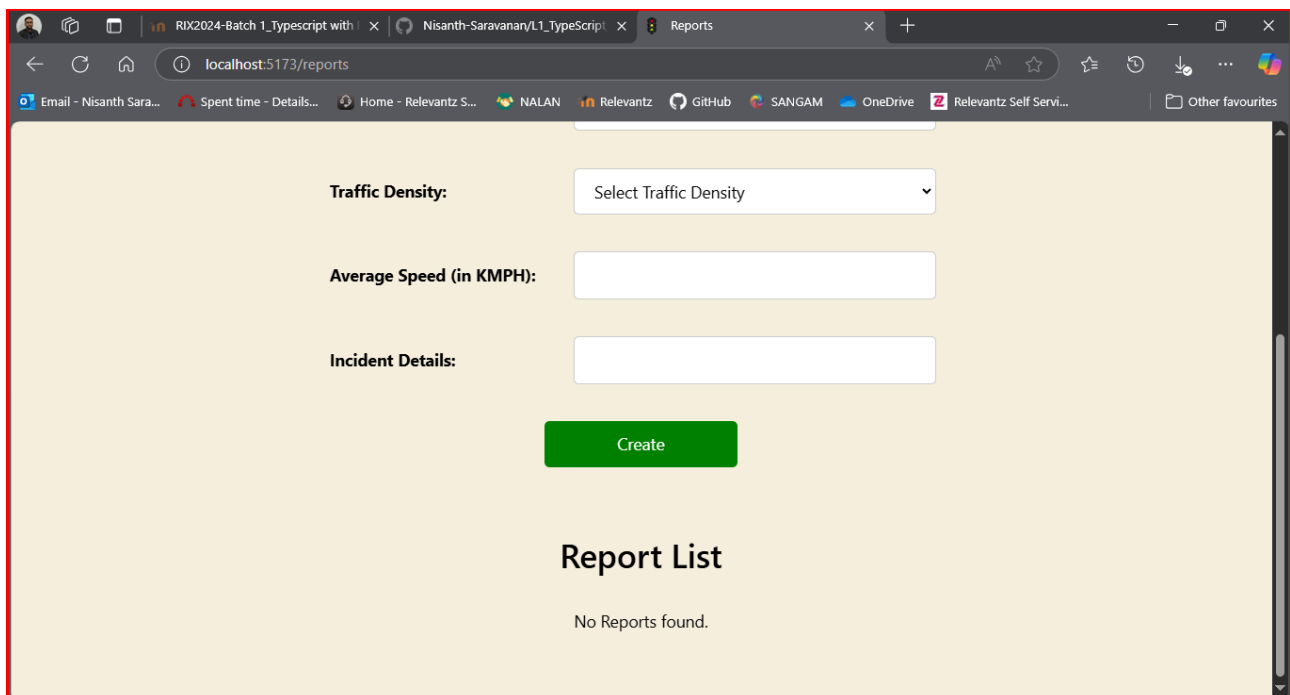


Reports Page:

No data exists in the JSON Database.



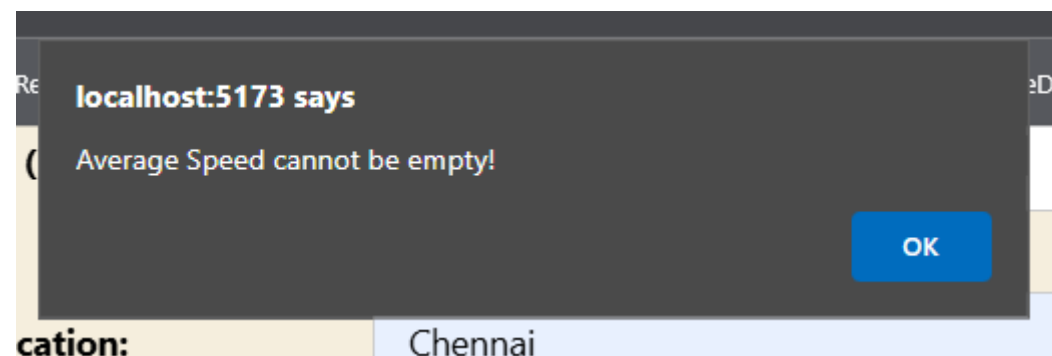
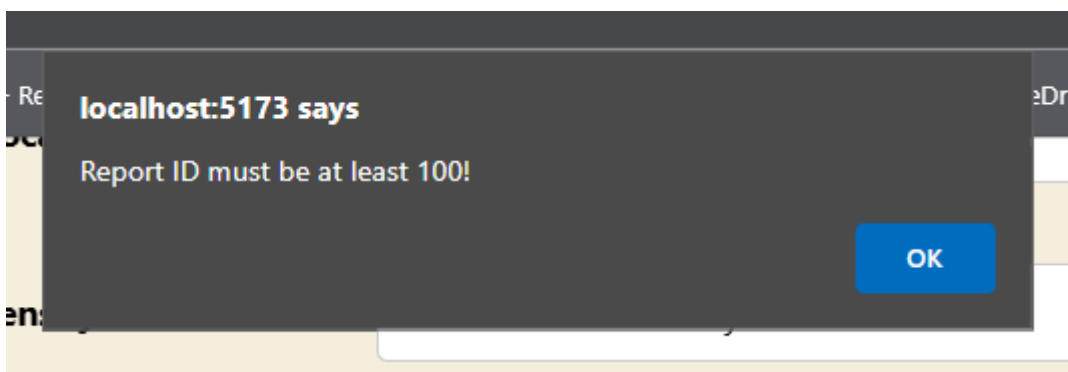
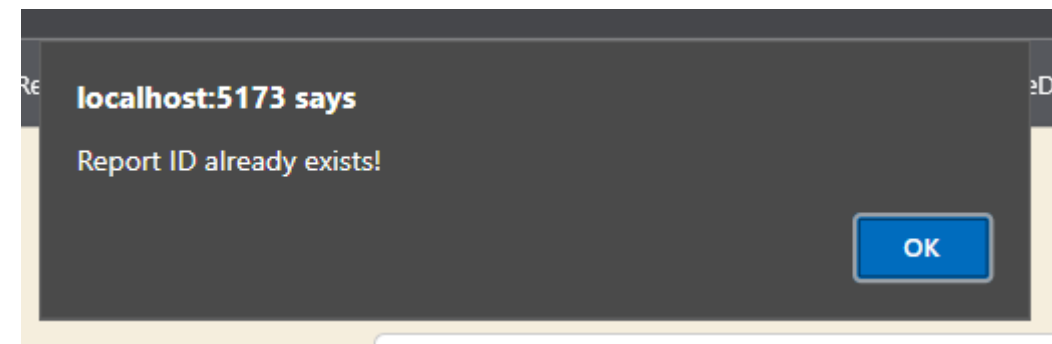
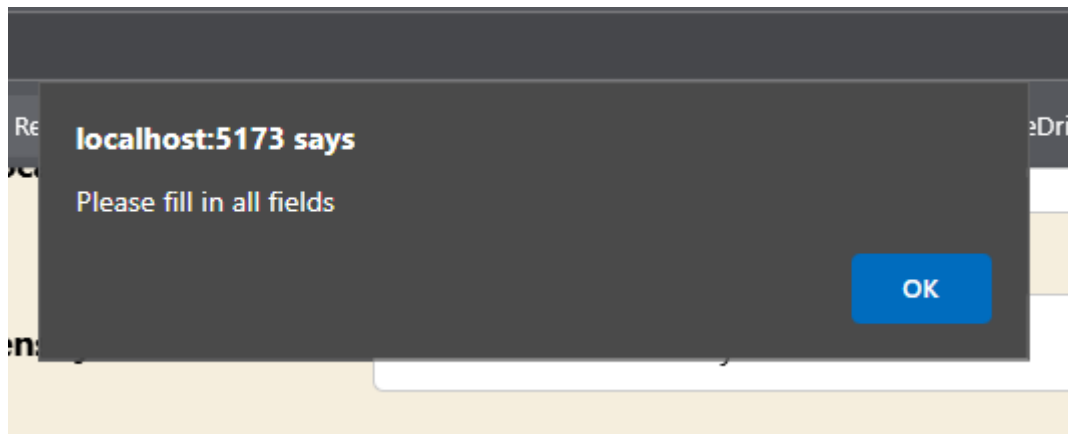
The screenshot shows a web browser window with the URL `localhost:5173/reports`. The page has a teal header with navigation links: "Traffic Reports", "Home", "Reports", and "About Us". The main content area is a light beige color and contains a "Report Form". The form has five input fields: "Report ID (in 100s):", "Report Location:", "Traffic Density:" (a dropdown menu showing "Select Traffic Density"), "Average Speed (in KMPH):", and "Incident Details:". The browser's address bar and tabs are visible at the top.

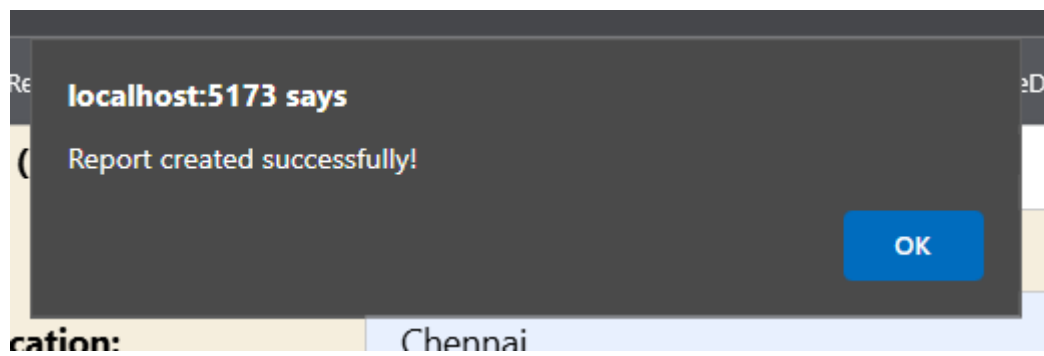
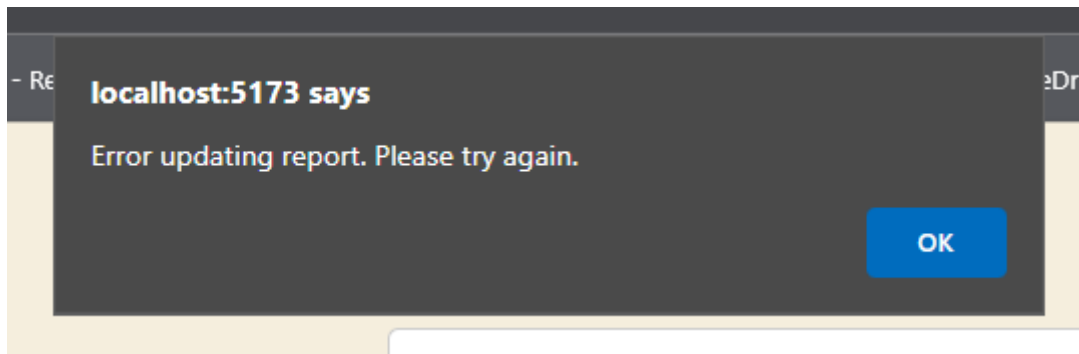
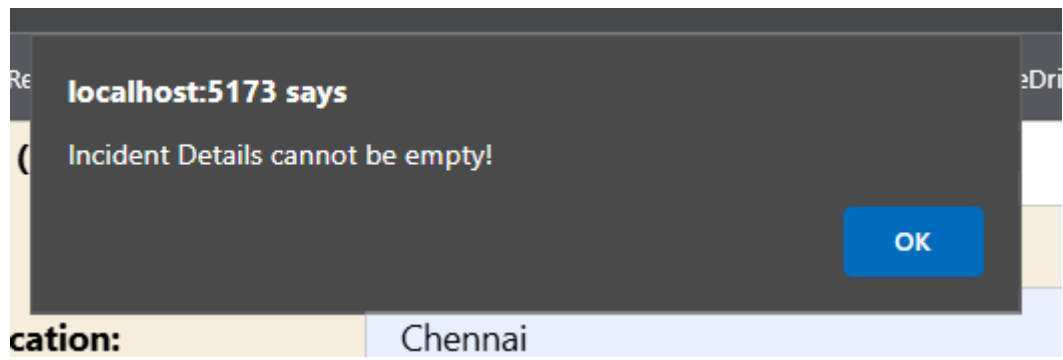


This screenshot shows the same web browser window, but the "Report Form" is partially obscured by a "Report List" section. The "Report List" section has the heading "Report List" and the text "No Reports found." below it. A green "Create" button is visible between the form and the report list. The "Traffic Density:" dropdown menu is still visible in the form section. The browser's address bar and tabs remain the same.

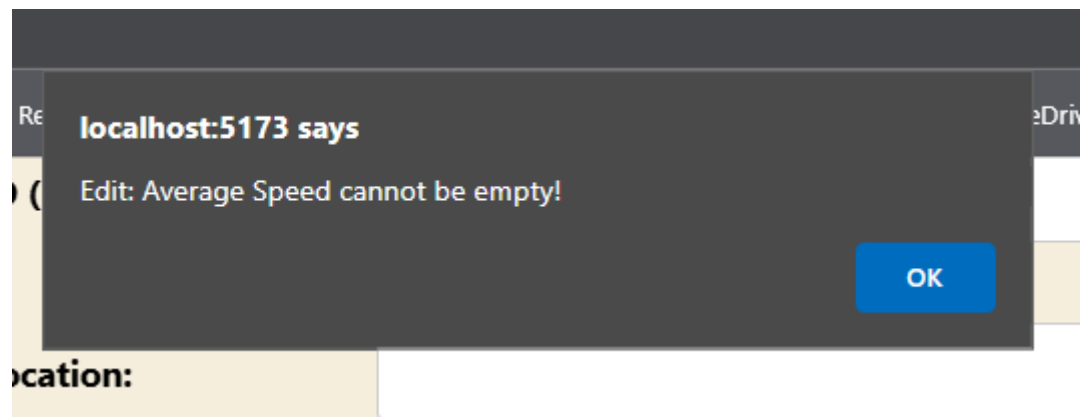
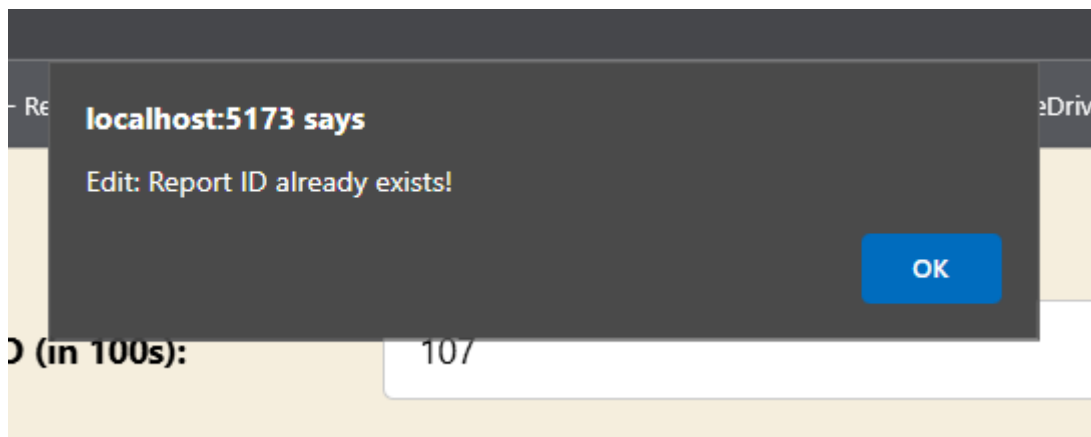
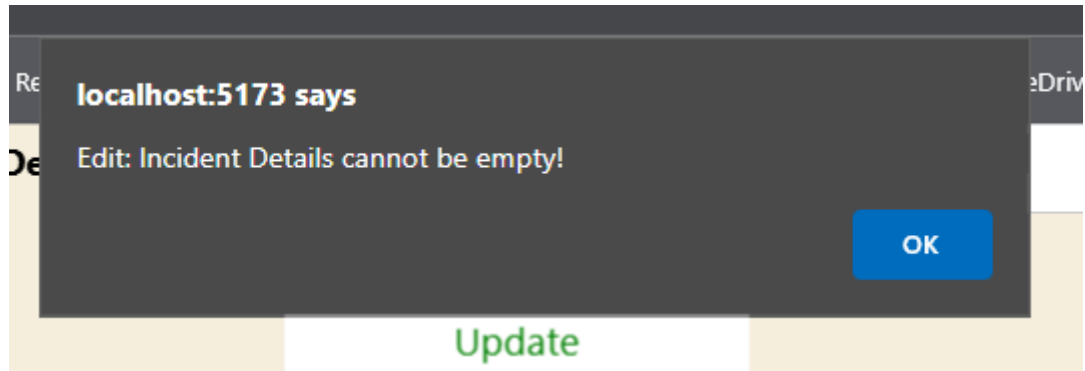
Report validations:

During Creation:





Edit Validations:



Report Creation:

localhost:5173/reports

Report List

Search

Report ID	Report Location	Traffic Density	Average Speed	Incident Details	Actions
121	Chennai	Heavy	121 KMPH	Lorem ipsum dolor sit amet.	Edit Delete
125	Chennai	Heavy	1233 KMPH	Lorem	Edit Delete

localhost:5000/TrafficReports

Pretty-print

```
[
  {
    "reportId": 121,
    "reportLocation": "Chennai",
    "trafficDensity": "Heavy",
    "averageSpeed": 121,
    "incidentDetails": "Lorem ipsum dolor sit amet.",
    "id": "a4e4"
  },
  {
    "id": "4a95",
    "reportId": 125,
    "reportLocation": "Chennai",
    "trafficDensity": "Heavy",
    "averageSpeed": 1233,
    "incidentDetails": "Lorem"
  }
]
```

Report Editing and Validations:

Report List					
Search					
Report ID	Report Location	Traffic Density	Average Speed	Incident Details	Actions
121	Chennai	Heavy	121 KMPH	Lorem ipsum dolor sit amet.	Edit Delete
125	Chennai	Heavy	1233 KMPH	Lorem	Edit Delete

Report Form

Report ID (in 100s):

131

Report Location:

Chennai, Tamil Nadu

Traffic Density:

Moderate

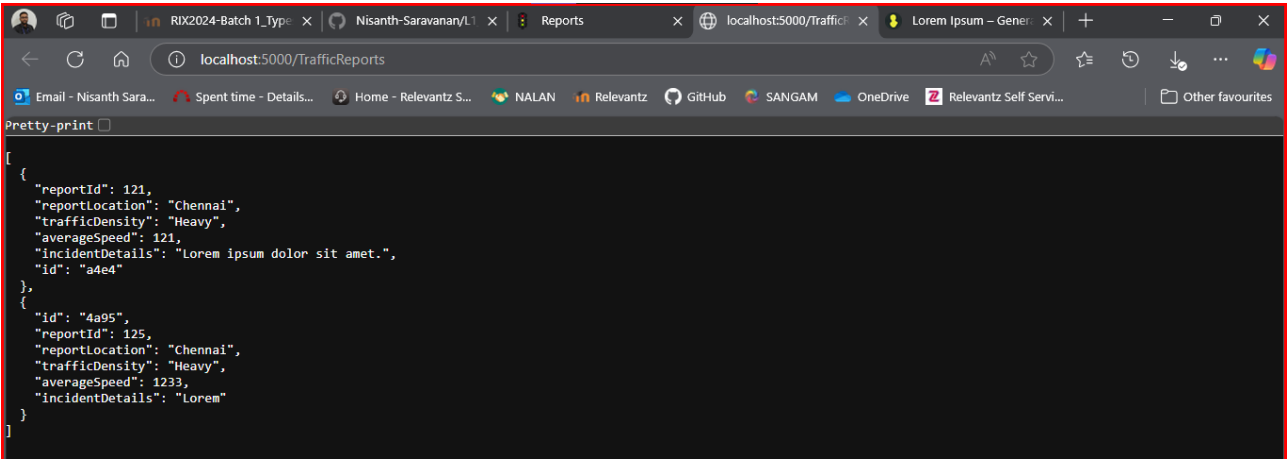
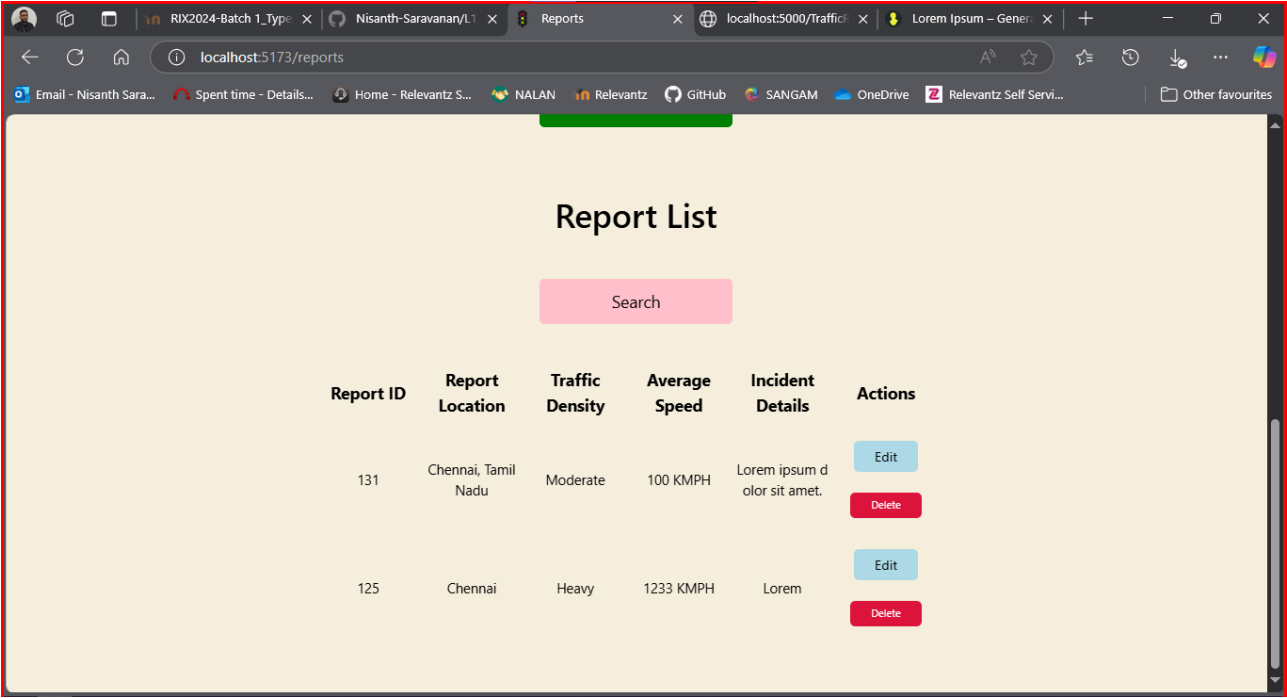
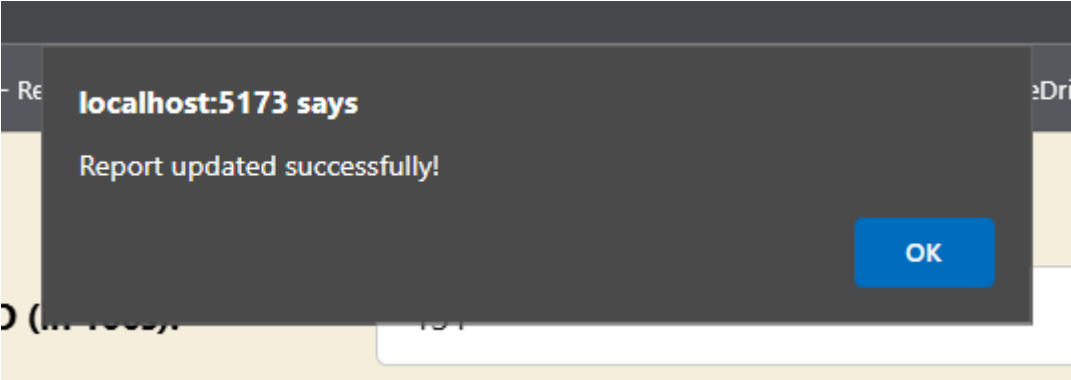
Average Speed (in KMPH):

100

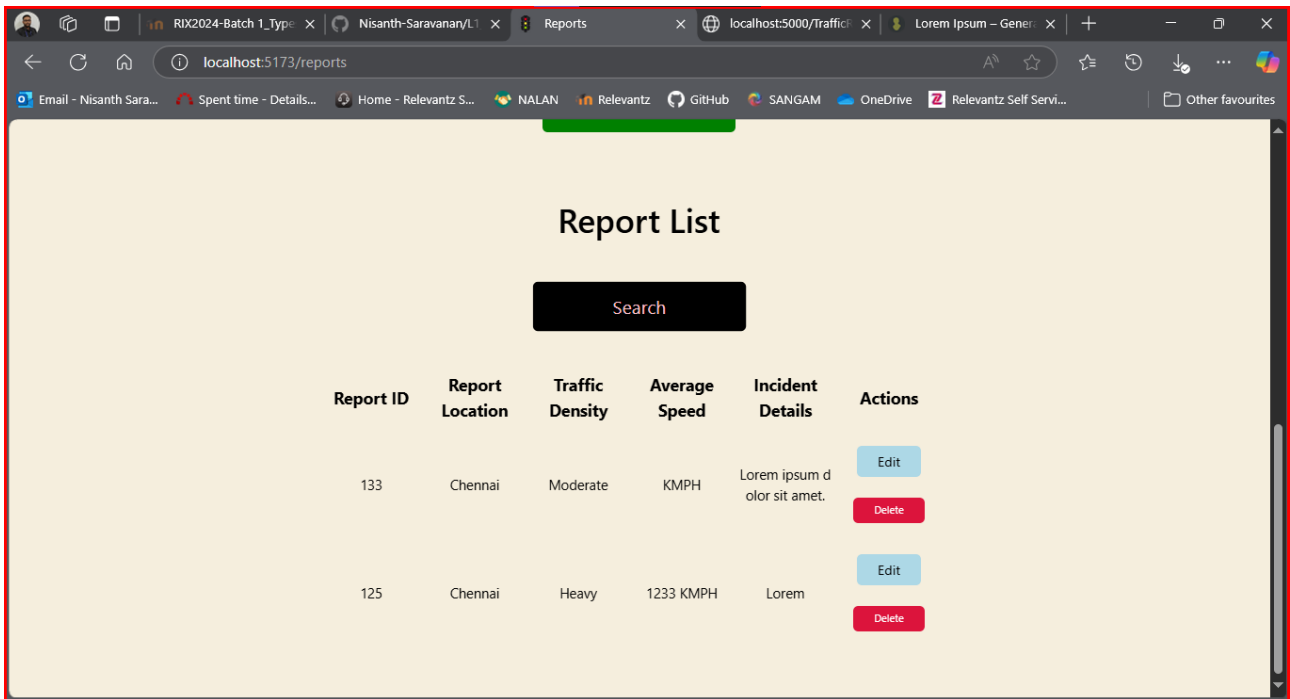
Incident Details:

Lorem ipsum dolor sit amet.

Update

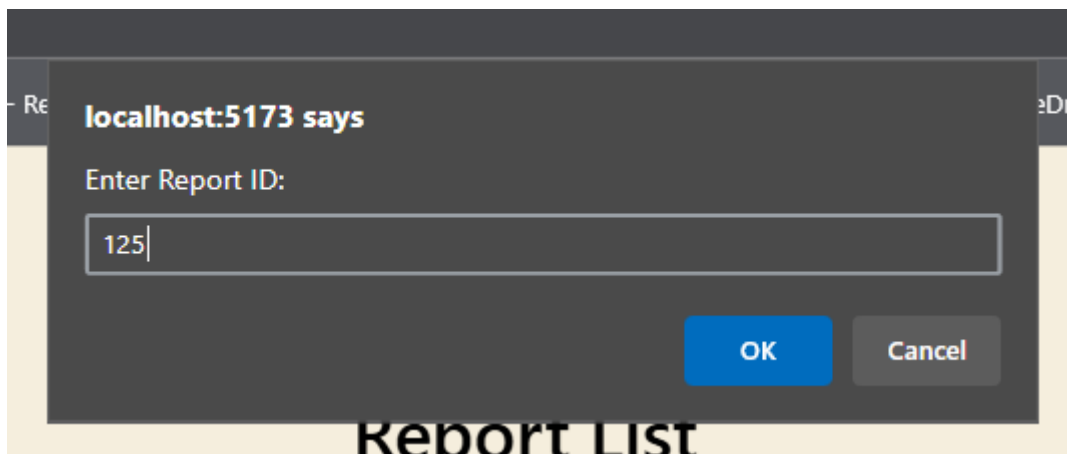


Deletion and Searching:

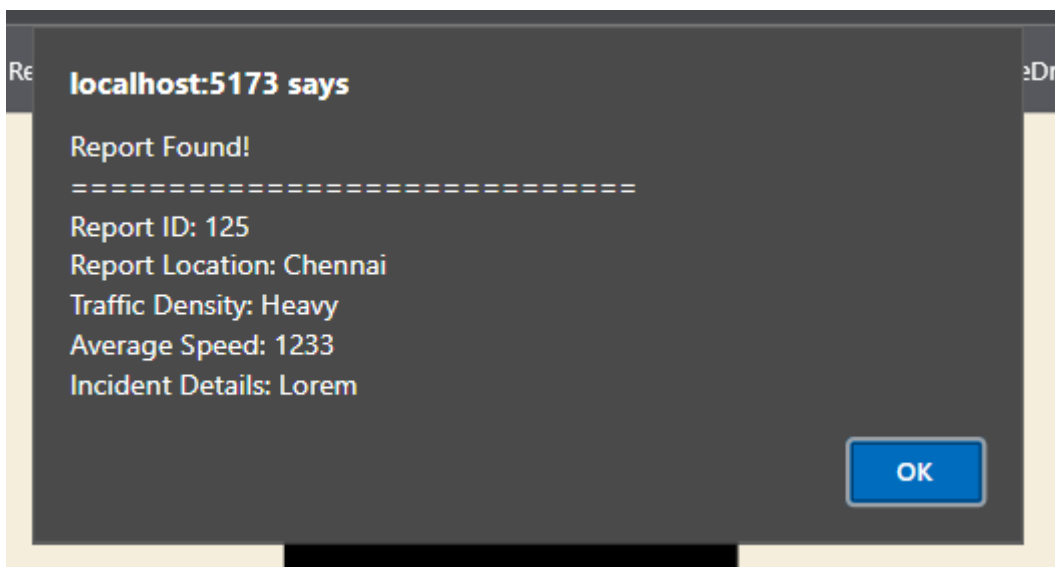


The screenshot shows a web browser window with the address bar displaying 'localhost:5173/reports'. The page title is 'Report List'. Below the title is a search bar with the placeholder text 'Search'. Below the search bar is a table with the following columns: Report ID, Report Location, Traffic Density, Average Speed, Incident Details, and Actions. The table contains two rows of data.

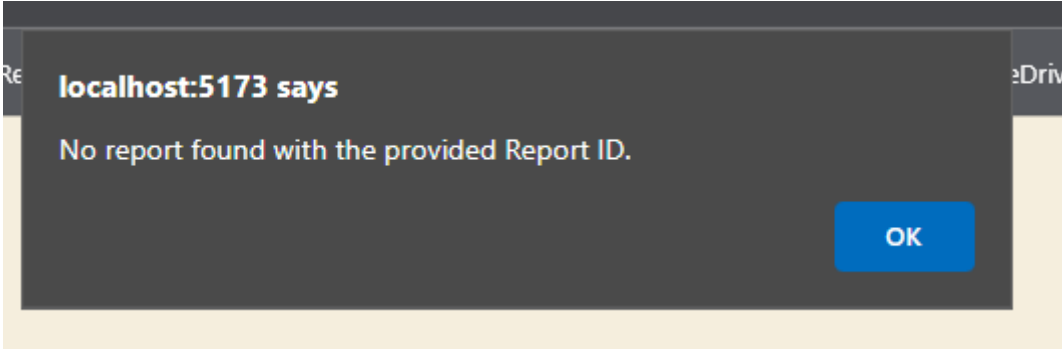
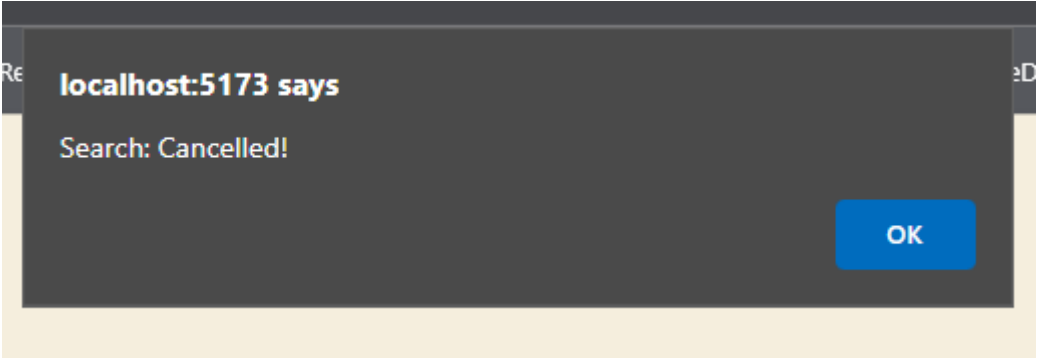
Report ID	Report Location	Traffic Density	Average Speed	Incident Details	Actions
133	Chennai	Moderate	KMPH	Lorem ipsum dolor sit amet.	<button>Edit</button> <button>Delete</button>
125	Chennai	Heavy	1233 KMPH	Lorem	<button>Edit</button> <button>Delete</button>



The screenshot shows a modal dialog box with the title 'localhost:5173 says'. The dialog contains the text 'Enter Report ID:' followed by a text input field containing the value '125'. At the bottom right of the dialog are two buttons: 'OK' and 'Cancel'.

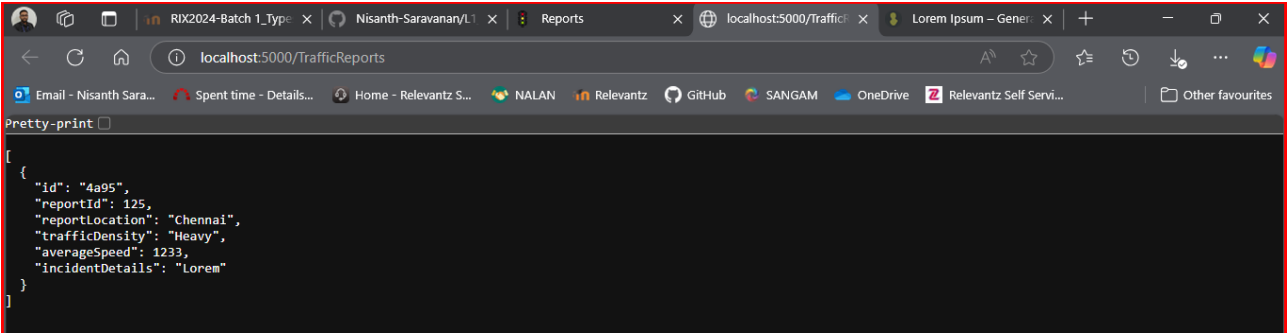


The screenshot shows a modal dialog box with the title 'localhost:5173 says'. The dialog contains the text 'Report Found!' followed by a separator line '====='. Below the separator line, the following details are displayed: 'Report ID: 125', 'Report Location: Chennai', 'Traffic Density: Heavy', 'Average Speed: 1233', and 'Incident Details: Lorem'. At the bottom right of the dialog is an 'OK' button.



Deletion:

Before:



Report List					
Search					
Report ID	Report Location	Traffic Density	Average Speed	Incident Details	Actions
125	Chennai	Heavy	1233 KMPH	Lorem	Edit Delete

After:

Report List

Search

Report ID	Report Location	Traffic Density	Average Speed	Incident Details	Actions
125	Chennai	Heavy	1233 KMPH	Lorem	Edit Delete

Traffic Density:

Select Traffic Density

Average Speed (in KMPH):

Incident Details:

Update

Report List

No Reports found.

